90	dark grey
91	light red
92	light green
93	yellow
94	light blue
95	light purple
96	turquoise
100	dark grey background
101	light red background
102	light green background
103	yellow background
104	light blue background
105	light purple background
106	turquoise background

Table 5: Additional colours

Now run the accompanying *lscolors.sh* (output shown in Figure 4) to see how to display listings of three files with some random extensions with customised colouring schemes. Again, once you find a colour scheme you are satisfied with, put the *LS_COLORS* setting in *~/.bashrc* to make it permanent.

You can explore ANSI escape sequences in detail at the links provided in the *References* section. Also, I encourage you to explore a package named *Bashish*, also in References. It is a theme environment for text terminals, and can change colours, fonts, transparency and background images on a per-application basis, according to its home page.

Console terminal manipulation with *tput*

Till now, we've used raw escape sequences, but now let's accomplish more tweaks with a utility called *tput*. This is a higher-level shell utility to control various aspects like text background, foreground, attributes, and to manipulate various cursor movements without issuing escape sequences. It simplifies escape-sequence operations, and provides a portable way to manipulate the capabilities of various console terminals. Table 6 has various *tput* arguments and their uses:

Argument	Capability
setab [0-7]	set background colour using ANSI escape value
setb [0-7]	set background colour
setaf [1-7]	set foreground colour using ANSI escape value
setf [1-7]	set foreground colour
bold	set bold mode
rev	set reverse mode
sgr0	turn off all attributes
cup y x	move cursor to x, y location (0, 0 is top left)
sc	save cursor position
rc	restore cursor
lines	get number of lines of the terminal
cols	get number of columns of the terminal
cub n	move n characters left
cuf n	move n characters right
clear	clear screen

Table 6: Tput arguments

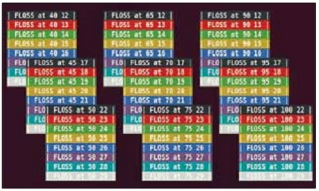

Figure 5: Overlapping rectangles formed by tput

The *setab* and *setaf* arguments support the same colour schemes as the ANSI escape sequences; the only difference is the index, ranging from 0-7. The colour index table for *setb* and *setf* is different, and is shown in Table 7.

Colour	Index
Black	0
Blue	1
Green	2
Cyan	3
Red	4
Magenta	5
Yellow	6
White	7

Table 7: Setb indexes

The output of the accompanying *tputops.py* (try it!) is shown in Figure 5. If you relate the appearance of overlapping rectangles to overlapping windows in the console graphics-mode applications that you often encounter in GNU/Linux, you are correct. Those applications work upon the basic concepts uncovered in this article. Explore the link for *tput* in References to create a full-fledged graphics menu-driven console application without any tool-kit.

See you next month, when we carry on! END

References

- ANSI escape code Wikipedia entry: *http://en.wikipedia.org/wiki/ANSI_escape_sequence*
- Bash Prompt HowTo: *http://www.gilesorr.com/bashprompt/howto/*
- Bashish theme environment: *http://bashish.sourceforge.net/*
- IBM DeveloperWorks *tput* article: *http://ibm.co/nVlHc*

By: Ankur Kumar Sharma

The author is a software developer and researcher. He likes to sing and play classic rock songs on the guitar, read self-help books, write, and explore all interesting things in his spare time. He blogs at *www.richnusgeeks.com*.